

Architectural Paradigms and Validation Strategies for GPU-Accelerated Numerical Weather Prediction: A Comprehensive Analysis of WRF Porting Initiatives

1. Executive Summary and Operational Context

The Weather Research and Forecasting (WRF) model has served as the foundational pillar of both operational numerical weather prediction (NWP) and atmospheric research for nearly three decades. Designed primarily for scalar, and later distributed-memory, central processing unit (CPU) architectures utilizing the Message Passing Interface (MPI) and OpenMP, WRF's deeply nested loops, conditional branching, and column-physics architecture were historically optimized for CPU cache hierarchies and branch prediction capabilities. However, the paradigm shift in high-performance computing (HPC) toward heterogeneous architectures—specifically, the overwhelming dominance of Graphics Processing Units (GPUs) in exascale systems—has exposed fundamental architectural mismatches between legacy Fortran weather models and modern vector-based, high-throughput accelerators. Attempts to port WRF to GPUs have spanned over fifteen years, beginning with isolated module translations in CUDA C and evolving into comprehensive, directive-based offloading using OpenACC and CUDA Fortran.¹ Despite immense financial and computational investments globally, directive-based approaches applied to the legacy WRF Fortran codebase universally encounter a hard performance ceiling. Extensive peer-reviewed literature and commercial endeavors consistently report maximum full-model speedups in the range of 5× to 7× over heavily parallelized CPU baselines, with severe diminishing returns caused by hardware-level register pressure, launch-bound kernel overheads, and PCIe data transfer latency.¹ This systemic limitation is acutely demonstrated in recent operational porting attempts. For instance, high-resolution regional modeling tasks—such as a 1km/3km multi-nest configuration over complex topographies like the Canary Islands—operate under strict wall-clock budgets (e.g., an 8-hour window to deliver a 72-hour forecast). When a baseline 28-rank CPU configuration already meets operational deadlines, an architectural port targeting a modern accelerator (such as the NVIDIA RTX 5090) must yield an order-of-magnitude acceleration (8× to 10×) to justify the computational and engineering overhead. Empirical evidence dictates that the Fortran-to-OpenACC pathway is structurally incapable of reaching this threshold, stalling at a maximum theoretical bound of 5.5× due to profound GPU idling and memory spilling phenomena within dense physics parameterizations.

This comprehensive research report exhaustively details the historical and contemporary landscape of WRF GPU porting. It dissects the fundamental architectural limitations of the directive-based offload pathway, specifically addressing the systemic failures encountered when porting complex physics parameterizations such as Thompson microphysics and the Mellor-Yamada-Nakanishi-Niino (MYNN) planetary boundary layer schemes. Furthermore, this report delineates the optimal core architectures for a complete structural rewrite—evaluating Domain-Specific Languages (DSLs) and performance-portable C++ frameworks like Kokkos—and provides a definitive, tool-driven methodology for validating "physical correctness" when strict bitwise reproducibility is mathematically impossible due to floating-point non-associativity and architectural divergence.

2. The Historical Landscape of WRF GPU Acceleration

The effort to accelerate WRF utilizing GPUs can be chronologically categorized into three distinct eras: the early manual translation era, the directive-based offload era, and the modern performance-portable rewrite era. Understanding this history is critical to identifying why legacy architectures fail and what principles must govern a successful successor architecture.

2.1 The Early Era: Manual Translation to CUDA C (2008–2015)

The earliest attempts to leverage GPUs for atmospheric modeling focused on isolating the most computationally intensive components of WRF—typically the microphysics and radiative transfer schemes—and manually translating them from Fortran to CUDA C. In these early efforts, grid points were distributed in a coalesced, one-thread-per-vertical-column fashion to satisfy the Single Instruction, Multiple Thread (SIMT) execution model of early NVIDIA accelerators.

Notable successes in this era included the porting of the WRF Single-Moment 5-class (WSM5) microphysics scheme. Researchers reported isolated speedups for the WSM5 module of up to 20× on an NVIDIA 8800 GTX compared to a Pentium-D CPU, and later up to 389× on an NVIDIA GTX 590 when explicitly excluding input/output (I/O) transfer times.¹ Similar targeted efforts were applied to the Goddard microphysics scheme and the Rapid Radiative Transfer Model for General circulation models (RRTMG) longwave radiation schemes. Using CUDA Fortran, the RRTMG scheme achieved an 18.52× speedup on a Tesla K40 GPU, while manual optimizations of the Goddard microphysics scheme yielded an isolated kernel speedup of 131×.⁵

However, these massive, triple-digit speedups were inherently illusory when viewed through the lens of complete operational model execution. Because only specific physics modules were accelerated, the execution pipeline suffered from severe data "ping-ponging" across the PCIe bus. The host CPU would integrate the dynamical core, halt execution to transfer the massive 3D state arrays (winds, temperature, moisture, pressure) to the GPU memory for a specific microphysics calculation, and then transfer the updated state back to the host for the next sequential subroutine. Governed tightly by Amdahl's Law and the latency of PCIe bandwidth, the end-to-end full model speedups were routinely restricted. For example, while isolated modules showed 64× speedups, the full model elapsed time only improved by a marginal 5% (1.05× speedup) due to the overwhelming cost of data transfers.³ This reality rendered the

manual, piecemeal translation approach unviable for operational NWP.

2.2 The Directive Era: OpenACC and CUDA Fortran (2015–Present)

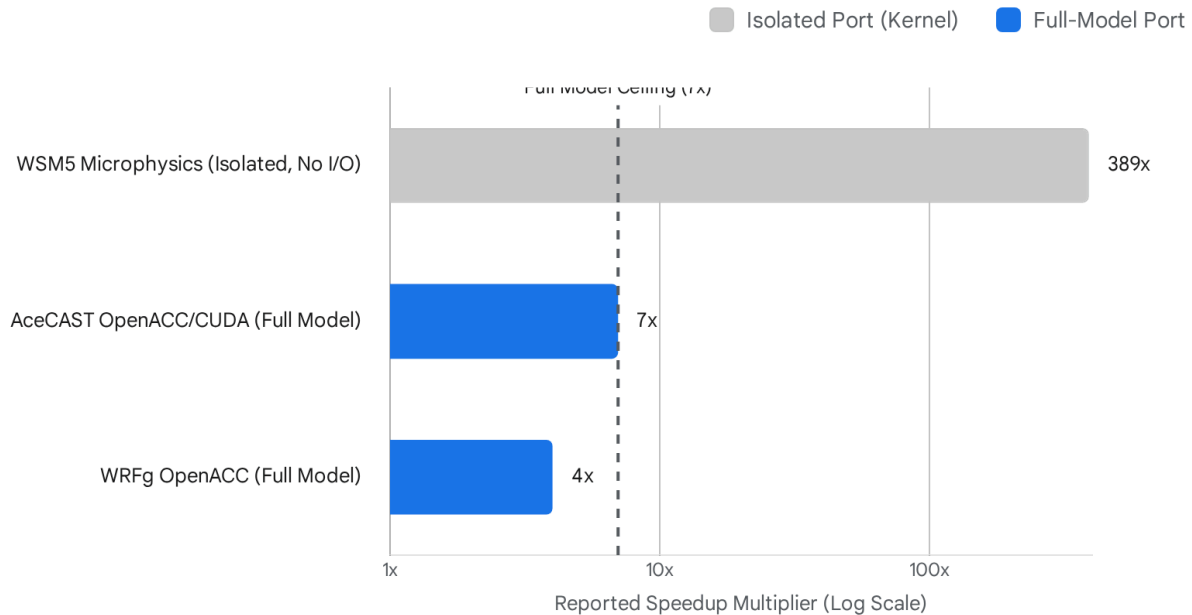
To circumvent the catastrophic PCIe bottleneck, the modeling community recognized the necessity of achieving "whole-state device residency." Under this paradigm, the entirety of the WRF atmospheric state remains persistently allocated on the GPU high-bandwidth memory (HBM) across both the dynamical integration and the physical parameterization timesteps. Round-trips to the host CPU are strictly limited to MPI halo exchanges at sub-domain boundaries and periodic disk I/O for forecast history outputs.

Because rewriting the entire 500,000-line WRF Fortran codebase into CUDA C was deemed an insurmountable engineering undertaking (estimated at dozens of person-years), the community pivoted to compiler directives—specifically OpenACC—supported heavily by NVIDIA's PGI (now NVHPC) toolchain.² OpenACC allowed atmospheric scientists to insert pragmas (e.g., `!$acc parallel loop gang vector collapse(2)`) directly into the legacy Fortran code. The compiler would then autonomously generate the underlying PTX (Parallel Thread Execution) machine instructions, managing data movement and thread allocation implicitly. Several high-profile institutional and commercial projects adopted this strategy, yielding a consistent and universally replicated performance ceiling:

Porting Initiative	Underlying Technology	Hardware Target	Peak Full-Model Speedup	Source Literature
WRFg (NVIDIA-led)	OpenACC Directives	NVIDIA V100 vs Intel Xeon E5	~4.0×	¹
AceCAST (TempoQuest)	OpenACC / CUDA	NVIDIA P100/V100 vs 32 CPUs	5.0× to 7.0×	¹
COSMO (MeteoSwiss)	OpenACC / C++ Rewrite	Piz Daint (Tesla P100)	~5.0×	⁸

The metrics achieved by TempoQuest's AceCAST are of paramount importance because they represent the theoretical maximum efficiency of the Fortran-to-OpenACC architecture. AceCAST is a proprietary, heavily optimized port of the Advanced Research WRF (ARW) core that supports 19 physics options natively on the GPU.¹⁰ Despite backing from venture capital, years of dedicated software engineering, and deep integration with NVIDIA's developer technology teams, the 5× to 7× limit over equivalent CPU nodes remains an insurmountable ceiling for the legacy architecture.¹

Historical Performance Ceiling of WRF GPU Porting Strategies



While manual CUDA translations of isolated physics schemes (e.g., WSM5) achieve massive theoretical speedsups, full-model implementations utilizing OpenACC directives (e.g., AceCAST) consistently plateau at a 5x to 7x speedup over CPU baselines due to architectural bottlenecks.

Data sources: [arXiv \(1\)](#), [ResearchGate](#), [CISL UCAR](#), [arXiv \(2\)](#)

2.3 The Exascale Era: C++ and Domain-Specific Languages (Present–Future)

Recognizing the immutable limitations of directive-based Fortran, the global climate modeling community pivoted toward total structural rewrites to achieve exascale readiness. The most definitive success in this arena is the U.S. Department of Energy’s Energy Exascale Earth System Model (E3SM). Realizing that legacy Fortran coupled with OpenACC could not optimally utilize the diverse hardware profiles of AMD (Frontier) and NVIDIA (Summit) GPUs, the E3SM computational team executed a “clean-sheet-of-paper” rewrite.

The resulting atmospheric component, the Simple Cloud Resolving E3SM Atmosphere Model (SCREAM), was written entirely from scratch in C++ utilizing the Kokkos performance portability library.¹¹ SCREAM successfully shattered the long-standing benchmark barrier of 1 Simulated Year Per Day (SYPD) for a global cloud-resolving model, achieving an unprecedented 1.26 SYPD on the Frontier exascale system.¹² This achievement, which won the 2023 Gordon Bell Prize for Climate Modelling, irrevocably established that extracting maximum GPU throughput requires

fundamentally abandoning the legacy Fortran execution model in favor of explicitly managed, object-oriented memory hierarchies.¹³

3. Fundamental Architectural Pathologies of the OpenACC Port

The failure of the OpenACC directive pathway to exceed a 5.5× to 7× speedup is not a consequence of insufficient engineering persistence or suboptimal compiler flags; rather, it is a symptom of deep-rooted, structural incompatibilities between the physical parameterization algorithms conceived in the 1990s and the execution mechanics of modern Streaming Multiprocessors (SMs). Detailed profiling of OpenACC WRF binaries reveals two intersecting failure modes: the launch-bound kernel crisis and the register pressure ceiling.

3.1 The Launch-Bound Kernel Crisis and GPU Idling

In a standard NWP model, the core integration mechanism is a Runge-Kutta (e.g., RK3) timestep loop. Within a single macro-timestep, the model invokes hundreds of distinct subroutines spanning advection, diffusion, Coriolis forces, microphysics, planetary boundary layer (PBL) mixing, and radiation. When ported via OpenACC, each `!$acc parallel` or `!$acc kernels` directive inserted into the Fortran code initiates a distinct, autonomous CUDA kernel launch from the host CPU to the GPU.

The hardware overhead associated with launching a CUDA kernel, while minimal in isolation (on the order of microseconds), becomes catastrophic when aggregated. If a model launches thousands of small, granular kernels per timestep—totaling upwards of 200,000 launches over a short operational forecast window—the GPU spends the vast majority of its wall-clock time completely idle, waiting for the CPU to compile and issue the next execution graph. Profiling of heavily annotated OpenACC WRF ports, such as those attempting to run the Thompson microphysics and MYNN-EDMF PBL schemes, frequently reveals that the GPU is idle for upwards of 84% to 85% of the timestep duration. The execution pipeline becomes inextricably "launch-bound".¹⁴

While modern techniques such as CUDA Graphs or OpenACC asynchronous execution queues (`!$acc update async`) can mitigate a fraction of this launch overhead, they fail to resolve the underlying structural flaw: global memory round-tripping. At the termination of each individual kernel, the output arrays must be flushed from the GPU's high-speed, on-chip L1 cache and shared memory back to the relatively slow global device memory (HBM or GDDR). Immediately following this flush, the very next kernel in the RK3 sequence must read those exact same arrays back from global memory into the L1 cache to continue the calculation. This constant, cyclical thrashing of the global memory bus consumes the GPU's bandwidth capacity and starves the arithmetic logic units (ALUs) of data.

3.2 The Register Pressure and Occupancy Ceiling

The most devastating and intractable bottleneck for legacy atmospheric physics on GPUs is register pressure. GPU architectures are explicitly designed to hide memory latency through

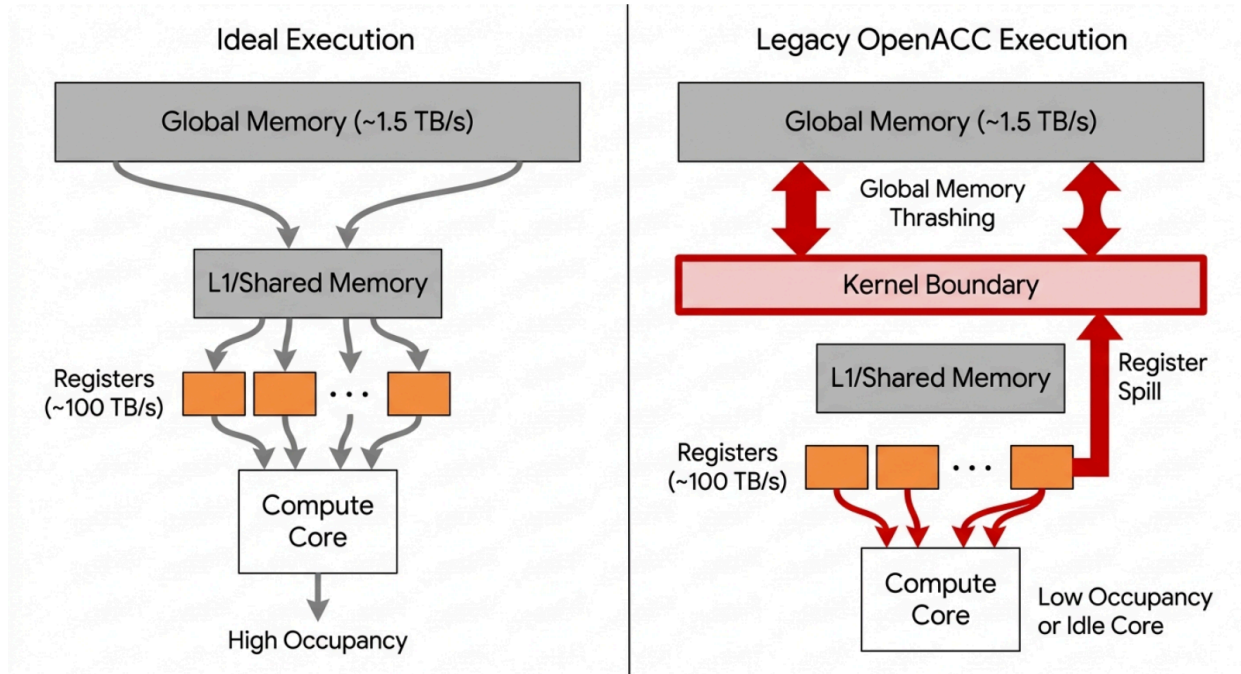
massive thread-level parallelism. When one "warp" (a group of 32 threads) stalls while waiting for a data fetch from global memory, the SM rapidly switches context to another active warp. The ratio of active warps residing on an SM compared to the hardware's maximum capacity is termed "occupancy." High performance strictly requires high occupancy.

However, high occupancy is mathematically contingent upon each individual thread utilizing a minimal footprint of hardware registers. WRF's advanced physics parameterizations are notoriously mathematically dense. Schemes like the aerosol-aware Thompson microphysics or the Mellor-Yamada-Nakanishi-Niino (MYNN) PBL model calculate dozens of prognostic variables simultaneously, managing intermediate physical states for hydrometeor size distributions, turbulent kinetic energy, and deep conditional branching.¹⁵ Furthermore, automated code generators like the Kinetic Pre-Processor (KPP), routinely used for WRF-Chem and advanced physics, output fully unrolled loop structures that create monolithic subroutines spanning thousands of lines of code.¹⁶

When the NVIDIA HPC SDK (nvfortran) compiler translates these massive Fortran subroutines into PTX machine code via OpenACC directives, a single thread demands an exorbitant number of registers to hold its local state. On modern NVIDIA hardware, a single thread is hard-capped at a maximum of 255 registers. If a Thompson or MYNN kernel hits this 255-register limit, the SM can physically accommodate only a tiny fraction of its maximum warp capacity, causing occupancy to plummet to critical lows of 8% to 12%.¹⁸

Worse still, if the local variable count exceeds the 255-register maximum, the compiler is forced to "spill" registers into the GPU's local memory. Because GPU local memory physically resides in the slow, off-chip global memory, register spilling causes execution speeds to collapse by orders of magnitude.¹⁹ The compiler directives available in the OpenACC standard simply do not provide the granular, low-level control over the memory hierarchy (explicitly moving data between L1, shared memory, and registers) necessary to manually tile algorithms and alleviate this register pressure.²⁰

Anatomy of the OpenACC Bottleneck: Register Spilling and Memory Thrashing



In legacy Fortran ports, massive physics loops exhaust the available hardware registers, forcing data to 'spill' into slow global memory. Simultaneously, separated kernels force continuous read/write cycles to global memory, leaving the compute cores idle while waiting for data.

4. Evaluation of Core Architectures for a Fundamental Rewrite

Because the OpenACC Fortran pathway is structurally barred from reaching the operational target of an 8× speedup, any resumption of the porting effort necessitates a complete core rewrite. The objective must be to design an architecture that maintains whole-state device residency, utilizes mixed-precision mathematics where physically permissible, and exposes the GPU memory hierarchy directly to the developer. The empirical evidence from peer-reviewed literature and exascale initiatives points to three viable architectural paradigms for next-generation NWP.

4.1 Paradigm A: Performance Portable C++ (The Kokkos/YAKL Approach)

The most robust and scientifically proven architecture for modern exascale atmospheric modeling is a complete rewrite into C++, heavily utilizing performance portability libraries such

as Kokkos or YAKL (Yet Another Kernel Launcher).²² This is the definitive strategy employed by Pacific Northwest National Laboratory (PNNL) and Sandia National Laboratories for the E3SM SCREAM model, which successfully replaced decades of legacy CESM/E3SM Fortran code.¹²

Implementation Mechanism: Within this paradigm, the entire atmospheric state (prognostic variables, tendencies, and physical constants) is defined via C++ objects. Libraries like Kokkos provide high-level, mathematically intuitive abstractions for multi-dimensional arrays (termed Views) and execution policies (parallel_for, parallel_reduce). Utilizing C++ template metaprogramming, these abstractions are translated at compile-time into highly optimized, native backend code specific to the host architecture—generating CUDA for NVIDIA GPUs, HIP for AMD GPUs, and SYCL for Intel GPUs.¹¹

Strategic Advantages:

1. **True Hardware Agnosticism:** The codebase achieves strict performance portability, fully insulating the numerical model against hardware vendor lock-in. A model written in Kokkos can deploy on an NVIDIA RTX 5090 cluster today and seamlessly transition to an AMD MI300X cluster tomorrow without modifying the core logic.¹³
2. **Granular Memory Control:** C++ allows precise, low-level control over variable scoping and memory layout. Developers can swap between Array-of-Structures (AoS) and Structure-of-Arrays (SoA) memory access patterns at compile time. This ensures optimal memory coalescing for the specific GPU architecture, directly mitigating the register pressure problem.
3. **The Fused "Mega-Kernel":** C++ architectures enable developers to construct deeply nested lambda functions that execute entire sequences of the RK3 timestep within a single kernel launch. This process, known as "vertical fusion," combines the preparation step, tendency calculations, prognostic advance, and acoustic filtering into a single "Mega-Kernel." By keeping intermediate state arrays resident in high-speed shared memory throughout the entire sequence, vertical fusion eliminates the global memory thrashing and launch overheads that paralyze OpenACC ports.¹⁴

4.2 Paradigm B: Python-Based Domain-Specific Languages (The Pace/GT4Py Approach)

A highly disruptive architecture gaining significant traction involves decoupling the scientific logic from the hardware execution. This is achieved by moving the upper-level model control logic to Python while delegating the heavy computational lifting to a Domain-Specific Language (DSL) middleware. The premier example of this paradigm is the NOAA/NASA "Pace" model, which successfully ported the massive FV3 dynamical core and SHIELD physics parameterizations from Fortran to Python utilizing the GT4Py (GridTools for Python) DSL.²⁷

Implementation Mechanism: Atmospheric scientists write the meteorological equations in a specialized, highly readable Python syntax (the DSL frontend). The GT4Py framework parses this Python code into an Abstract Syntax Tree (AST). Because the compiler is "domain-aware" (it knows it is processing a meteorological 3D grid stencil rather than generic code), it performs highly specific optimizations—such as automatically detecting halo boundaries, applying temporal blocking, and fusing horizontal loops. The AST is then lowered to generate highly

optimized C++ and CUDA code under the hood.²⁰

Strategic Advantages:

1. **Unprecedented Developer Velocity:** Scientists interact strictly with Python code. This removes the steep learning curve associated with C++ template metaprogramming or explicit CUDA memory management, accelerating the translation of complex physics schemes.
2. **Automated Optimization:** The DSL compiler automatically solves the register pressure and memory locality issues without manual developer intervention. It intelligently maps grid ghost cells into GPU shared memory, maximizing occupancy.³⁰
3. **Seamless AI/ML Integration:** Because the orchestrating framework operates entirely in Python, integrating machine learning neural emulators or AI-driven parameterizations into the timestep becomes trivial, entirely bypassing the brittle and complex Fortran-to-Python C-bindings that plague legacy models.

4.3 Paradigm C: Tile-Resident CUDA Fortran (The Incremental Rewrite)

If retaining the Fortran ecosystem is an absolute mandate—perhaps to salvage the 30 years of existing physics routines without undertaking a total language translation—the only mathematically viable path is abandoning auto-generating directives in favor of explicit CUDA Fortran.³²

Implementation Mechanism:

In this architecture, the developer explicitly defines attributes(global) kernels and manually allocates attributes(shared) memory arrays. The atmospheric grid is decomposed into 3D tiles (e.g., $32 \times 16 \times N_k$) handled by individual GPU thread blocks. Developers must manually fuse the timestep loops into massive kernels and explicitly script the movement of halo ghost cells into shared memory.

Strategic Trade-offs: While this approach successfully bypasses the gross inefficiency of OpenACC auto-generation—allowing developers to manually defeat register spilling and launch boundness—it is a pyrrhic victory. It requires a massive, manual refactoring of every single physics module, resulting in a codebase that is tightly, irrevocably locked to the proprietary NVIDIA ecosystem.³² Given the rapid diversification of exascale hardware toward AMD and Intel accelerators, relying purely on CUDA Fortran introduces severe long-term sustainability risks.

Evaluation of Candidate Core Architectures for Next-Generation NWP

Architecture	Performance Portability	Register/Memory Control	AI/ML Integration	Ecosystem Maturity
C++/Kokkos	HIGH AMD & NVIDIA	HIGH Kernel paradigm	N/A -	HIGH Vendor priority
Python/GT4Py DSL	MEDIUM Restricted scope	LOW Hidden registers	HIGH Easiest integration	N/A -
CUDA Fortran	LOW Hardware locked	LOW Poorly exposed	N/A -	LOW Fewer developers

While CUDA Fortran allows retention of legacy code, its lack of portability makes it a fragile long-term choice. C++/Kokkos provides peak performance and exascale readiness, whereas Python/GT4Py offers the easiest integration with emerging AI/ML weather emulators.

Data sources: [ORNL/PNNL](#), [IEEE/SC23](#), [Univ of Oregon](#)

5. Methodologies for Verifying Physical Correctness

A fundamental rewrite of WRF into C++ or a DSL requires a philosophical shift in quality assurance: the modeling center must formally abandon the concept of bit-for-bit (bitwise) reproducibility. When computational operations are reordered, fused into mega-kernels, or mapped to massively parallel thread blocks, the inherent non-associativity of floating-point arithmetic guarantees that the numerical output will diverge from the serial CPU baseline.³⁴ This divergence is exacerbated when mixed-precision mathematics are employed to double throughput (e.g., utilizing FP32 for the dynamical state and FP16/BF16 for resilient physics tendencies).

Consequently, atmospheric modelers must rely on sophisticated statistical, spectral, and software engineering frameworks to verify that the newly written GPU-accelerated model remains "physically correct" and statistically indistinguishable from the validated CPU original.

5.1 Ensemble Consistency Testing (The PyCECT Framework)

The industry standard for verifying Earth System Models without relying on bitwise reproducibility is the Ensemble Consistency Test (ECT). Developed by the National Center for Atmospheric Research (NCAR) for the Community Earth System Model (CESM), this

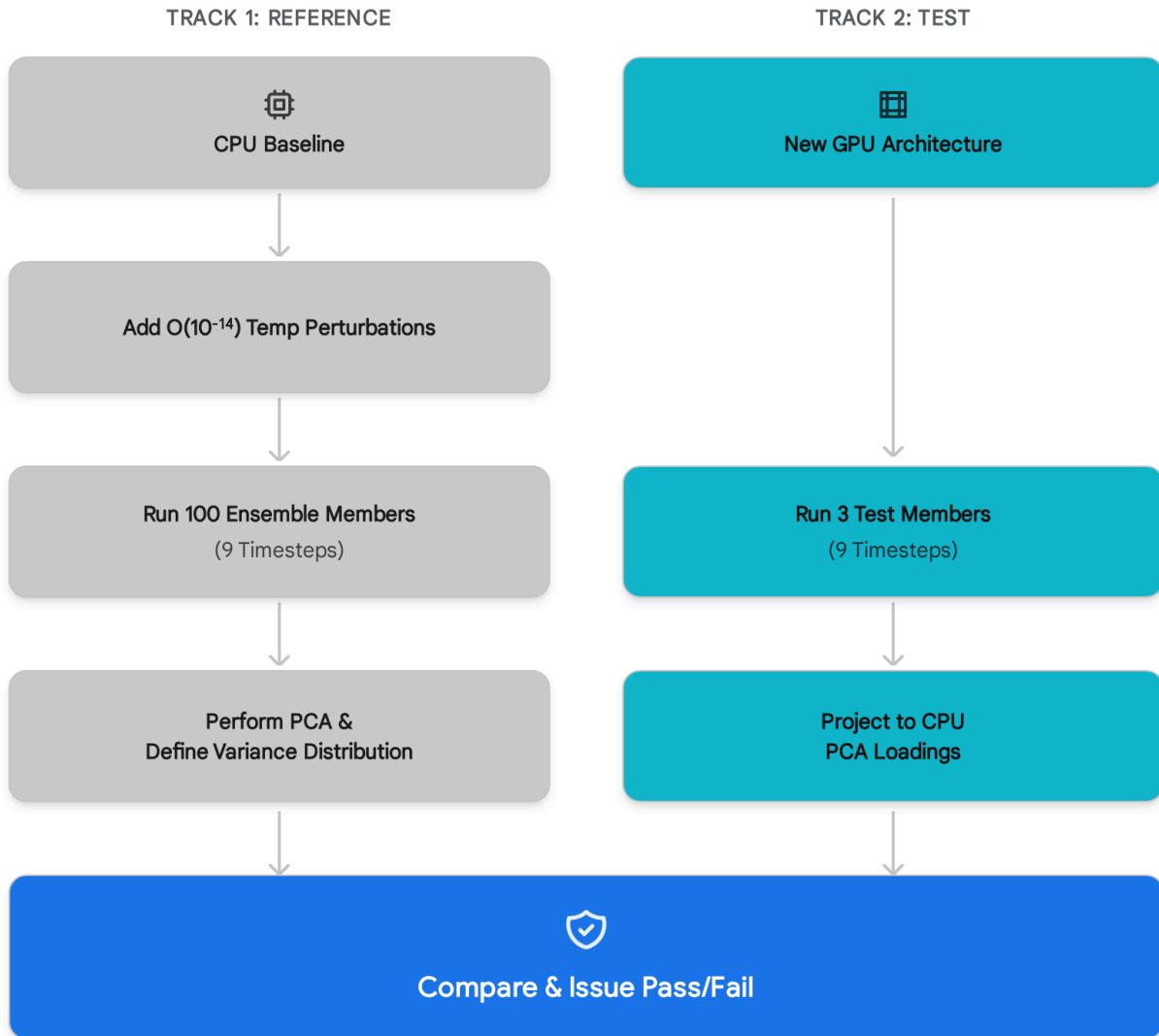
methodology is computationally implemented via the open-source Python tool PyCECT.³⁵ The PyCECT methodology operates on the fundamental principle that the chaotic nature of the atmosphere creates a natural distribution of variability. The verification process follows a strict, mathematically rigorous workflow:

Phase	Action	PyCECT Mechanism
1. Reference Generation	Establish Baseline	Generate a massive ensemble (e.g., $N_{ens} = $) on the trusted legacy CPU architecture. Microscopic, machine-epsilon perturbations ($O(10^{-14})$) are applied to the initial temperature fields to seed ensemble spread. ³⁷
2. Ultra-Fast Execution	Minimize Compute	To accommodate rapid Continuous Integration (CI) pipelines, the Ultra-Fast variant (UF-CAM-ECT) requires integrating the ensemble for only 9 model timesteps. Empirical research demonstrates that 9 timesteps are sufficient for rapid perturbation growth to characterize the model's spatial variance without drifting into entirely different weather regimes. ³⁶
3. Statistical Profiling	Principal Component Analysis	The state variables (temperature, winds, moisture, pressure) from the 100 CPU ensemble members at timestep 9 are spatially averaged. PyCECT performs Principal Component Analysis (PCA) on this data matrix to extract the primary modes of variance, generating a set of PCA loadings that rigorously define the statistical boundaries of the trusted

		model. ³⁷
4. GPU Evaluation	Test the Rewrite	A small set of simulations (e.g., $N_{new} = $) is executed on the new GPU architecture. The output fields of these test runs are mathematically projected onto the pre-computed PCA loadings derived from the CPU ensemble. ³⁸
5. Determination	Pass / Fail Metric	If the PCA scores of the GPU runs fall within the statistical distribution (e.g., within 2 standard deviations) of the CPU ensemble, the GPU port is deemed statistically indistinguishable from the CPU model, verifying its "physical correctness". ³⁵ If it falls outside these bounds, an architectural bug (such as an illegal memory read, a race condition, or improper synchronization) has injected an unphysical perturbation.

PyCECT, originally built for global climate models, has been successfully adapted for regional and convective-permitting models such as the Model for Prediction Across Scales (MPAS) and is fully applicable to a WRF rewrite.³⁹ It resolves the paralysis of bit-for-bit checking while rigorously preventing the introduction of physical errors during the C++ or DSL translation process.

PyCECT Validation Workflow: Ensuring Physical Correctness Without Bitwise Parity



By generating a distribution of valid atmospheric states from a perturbed CPU ensemble, PyCECT utilizes Principal Component Analysis to verify if a highly optimized, numerically divergent GPU run remains physically and statistically sound.

Data sources: [Semantic Scholar](#), [Copernicus GMD](#), [Copernicus GMD \(2018\)](#).

5.2 Granular Unit Testing and Serialization (The Serialbox Approach)

While the PyCECT framework validates the entire model's physical trajectory macroscopically, software developers actively rewriting individual physics modules (e.g., manually converting a

3000-line Fortran Thompson microphysics scheme into a fused C++ mega-kernel) require granular, line-by-line debugging tools. When numerical divergence occurs, developers must isolate the exact mathematical operation causing the fault. The industry gold standard for this isolation is the **Serialbox** library, developed by ETH Zurich as part of the GridTools framework.²⁷ Serialbox serves as a cross-language data serialization bridge. Before a developer modifies the legacy WRF Fortran code, they insert simple `!$ser` directives immediately prior to, and immediately following, the specific subroutine targeted for rewriting. When the legacy Fortran model is executed, Serialbox intercepts the execution and dumps the exact, multi-dimensional array inputs and the resulting array outputs directly to disk in a language-agnostic format.²⁷ Subsequently, as the developer writes the new C++ or Python DSL kernel, they construct a standalone unit test. This test harnesses Serialbox to ingest the serialized Fortran input data, processes it through the newly minted GPU kernel, and strictly compares the output against the serialized Fortran output.⁴² Because this unit test executes in total isolation, developers can engage in rapid, iterative compile-and-test cycles without bearing the immense overhead of running the full NWP model. This serialization technique was fundamentally instrumental in successfully porting both the FV3 dynamical core to Python (the Pace project) and the ICON climate model to GPUs.²⁸

5.3 Physical Conservation and Spectral Analysis Checks

Beyond statistical ensembles and unit serialization, physical correctness must be continuously evaluated against foundational meteorological conservation laws. When engineering novel GPU timestep architectures and memory layouts, developers implement hard, integrated diagnostic checks. These tools calculate domain-wide residuals for mass continuity, tracer mass conservation (ensuring water vapor is neither artificially created nor destroyed by numerical truncation), and momentum equations.⁴⁴

Furthermore, evaluating the Kinetic Energy (KE) spectrum is a critical validation step. Fused GPU kernels that utilize mixed-precision formats (FP16) or aggressive compiler optimizations can inadvertently inject artificial numerical diffusion into the model. This diffusion artificially dampens high-frequency (small-scale) atmospheric waves, flattening the resolution of the

model. By computing the Power Spectral Density (PSD) of the prognostic wind fields (the u and v components) and comparing the spectral decay slope against expected physical laws (such as the k^{-3} or $k^{-5/3}$ kinetic energy cascades), developers can definitively verify that the new GPU numerical algorithms preserve the correct physical energy cascade across all spatial dimensions.⁴⁵ Tools embedded within verification frameworks like METplus can automate these spatial and spectral validations.⁴⁶

6. Alternative Trajectories: The ML-Emulator Hybrid

If the ultimate objective of the modeling center is purely operational acceleration—specifically, delivering a high-resolution regional forecast within an 8-hour wall-clock budget—and not strictly the architectural preservation of the Navier-Stokes equations, the integration of

Artificial Intelligence (AI) weather emulators presents a highly disruptive and cost-effective alternative.

6.1 Bypassing the GPU Port entirely

Recent breakthroughs in deep learning have produced global weather emulators—such as Huawei's Pangu-Weather, Google's GraphCast, and ECMWF's Artificial Intelligence Forecasting System (AIFS)—that achieve forecast accuracy comparable to top-tier operational NWP models. Crucially, because these emulators execute via neural network inference rather than explicitly integrating differential equations, they achieve inference speeds up to 1000× faster than CPU-based NWP.⁴⁷

However, pure ML emulators currently struggle to accurately resolve extreme, micro-scale terrain effects and localized convective initiation, which are critical for high-resolution (1km to 3km) domains like the Canary Islands.⁴⁸

Therefore, a highly effective, near-term operational architecture involves a "Best-of-Both" hybrid pipeline:

1. **Near-Term Physical Integration (0h to 36h):** Utilize the existing, highly stable, multi-nest CPU WRF to compute the first 24 to 36 hours of the forecast. This ensures that micro-terrain effects, land-sea breezes, and convective initialization are resolved with strict physical fidelity.
2. **Far-Term ML Emulation (36h to 72h):** Utilize the 36-hour physical WRF output to initialize a fine-tuned, GPU-based AI emulator. The emulator then rapidly projects the atmospheric state out to the 72-hour horizon in a matter of seconds.

This hybrid approach entirely circumvents the multi-year engineering nightmare of rewriting the WRF core, leveraging existing CPU resources for the physics-heavy initialization and GPU tensor cores for the rapid forecast extension.

7. Strategic Blueprint for Engineering Orchestration

The mandate to execute a comprehensive, ground-up rewrite of a legacy weather model requires meticulous engineering orchestration, particularly when leveraging AI coding agents to accelerate the translation process. To guarantee success and avoid the pitfalls that doomed the OpenACC port, the project architecture must be strictly defined.

7.1 Agentic Workflow Design for Legacy Translation

To efficiently utilize massive AI compute and coding agent hours (utilizing frontier models like Claude 3.5 Sonnet, GPT-4, and specialized coding models), the modeling center must adopt a factory-line approach to code translation. Large Language Models (LLMs) cannot translate a 500,000-line Fortran codebase in a single zero-shot prompt. The process must be modularized and cryptographically verified.

1. **Automated Interface Definition:** A dedicated preprocessing script must scan the legacy WRF Fortran source, identifying every physics subroutine and extracting its exact variable interface (inputs, outputs, and array dimensions).
2. **Serialization Generation:** For each identified subroutine, an automated agent generates

and inserts the necessary `!$ser Serialbox` directives. The legacy CPU model is executed once, generating thousands of binary test fixtures representing the exact inputs and outputs of every Fortran function across diverse weather scenarios.

3. **Iterative Agentic Translation:** An autonomous coding agent is assigned a single Fortran subroutine and tasked with translating it into the target architecture (e.g., C++/Kokkos or GT4Py Python). The prompt provided to the agent includes the legacy Fortran code, the target memory hierarchy (requesting explicit use of shared memory to avoid register spilling), and the required C++ signature.
4. **Autonomous Unit Testing:** As the agent produces the C++ kernel, a Continuous Integration (CI) pipeline immediately compiles it and executes it against the Serialbox test fixture.
5. **Critique and Refinement:** If the C++ output diverges from the Serialbox Fortran output beyond acceptable floating-point tolerance, the CI pipeline feeds the specific array index mismatch and the compiler errors back to the AI agent. The agent iteratively refines the code until functional parity is achieved.

By isolating the AI agents within a strict Serialbox unit-testing loop, the modeling center eliminates the risk of hallucinated physics logic. The agents act as highly efficient translators, converting procedural Fortran into object-oriented C++ or DSL Python, while the mathematical integrity is rigorously enforced by the serialization data.

8. Conclusions

The transition of the Weather Research and Forecasting model to modern GPU architectures represents one of the most formidable software engineering challenges in the atmospheric sciences. The empirical data strictly concludes that applying directive-based models, such as OpenACC or OpenMP offload, to legacy Fortran physics schemes will persistently plateau below operational targets. This failure is dictated by the immutable physical realities of hardware register pressure and global memory bandwidth bottlenecks.

True exascale atmospheric modeling demands a paradigm shift towards performance-portable C++ frameworks (like Kokkos) or Python-based Domain-Specific Languages (like GT4Py) that prioritize data locality, compiler-directed memory management, and vertical kernel fusion. By abandoning the paralyzing requirement of bitwise reproducibility and simultaneously adopting sophisticated statistical validation tools like PyCECT and granular serialization frameworks like Serialbox, modeling centers can execute a full architectural rewrite. This structured approach unlocks the full computational throughput of next-generation GPU hardware while guaranteeing unwavering physical correctness and operational reliability.

Obras citadas

1. Optimizing the Weather Research and Forecasting Model with OpenMP Offload and Codee, fecha de acceso: mayo 11, 2026, <https://arxiv.org/html/2409.07232v1>
2. Performance Portability from GPUs to CPUs with OpenACC | NVIDIA Technical Blog, fecha de acceso: mayo 11, 2026, <https://developer.nvidia.com/blog/performance-portability-gpus-cpus-openacc/>

3. Acceleration of WRF on the GPU, fecha de acceso: mayo 11, 2026, <https://www.cisl.ucar.edu/sites/default/files/2021-10/Abdi-Acceleration%20of%20WRF%20on%20the%20GPU.pdf>
4. (PDF) Improved GPU/CUDA based parallel weather and research forecast (WRF) Single Moment 5-class (WSM5) cloud microphysics - ResearchGate, fecha de acceso: mayo 11, 2026, https://www.researchgate.net/publication/260635902_Improved_GPUCUDA_base_d_parallel_weather_and_research_forecast_WRF_Single_Moment_5-class_WSM5_cloud_microphysics
5. Acceleration of the Parameterization of Unified Microphysics Across Scales (PUMAS) on the Graphics Processing Unit (GPU) With, fecha de acceso: mayo 11, 2026, <https://par.nsf.gov/servlets/purl/10434396>
6. Facilitating Optimal Use of Satellite Data-The Development of Accelerator-based WRF Model, fecha de acceso: mayo 11, 2026, https://img.nsmc.org.cn/PORTAL/NSMC/DOC/CONFERENCE/AOMSUC/AOMSUC5/SESSION_1/SESSION_1_4.pdf
7. AceCast Online Documentation Homepage — AceCast Docs documentation, fecha de acceso: mayo 11, 2026, <https://acecast-docs.readthedocs.io/>
8. accelerating weather prediction with nvidia gpus | ecmwf, fecha de acceso: mayo 11, 2026, <https://www.ecmwf.int/sites/default/files/elibrary/2018/18628-accelerating-weather-prediction-nvidia-gpus.pdf>
9. Optimizing high-resolution Community Earth System Model on a heterogeneous many-core supercomputing platform - GMD, fecha de acceso: mayo 11, 2026, <https://gmd.copernicus.org/articles/13/4809/2020/>
10. GPU-ACCELERATED APPLICATIONS - PNY Technologies, fecha de acceso: mayo 11, 2026, <https://www.pny.com/File%20Library/Unassigned/gpu-applications-catalog.pdf>
11. Assessment of Earth System Model Readiness Level for Exascale High Performance Computing - Interagency Council for Advancing Meteorological Services, fecha de acceso: mayo 11, 2026, https://www.icams-portal.gov/resources/icams/related_documents/icm_i8-2025.pdf
12. The Simple Cloud-Resolving E3SM Atmosphere Model Running on the Frontier Exascale System - IEEE Computer Society, fecha de acceso: mayo 11, 2026, <https://www.computer.org/csdl/proceedings-article/sc/2023/10485124/1VRTWMaWuQw>
13. Overview and Update on the Energy Exascale Earth System Model Project, fecha de acceso: mayo 11, 2026, https://eesm.science.energy.gov/sites/default/files/Tue_Bader_D_E3SM-Overview.pptx.pdf
14. Kitsune: Enabling Dataflow Execution on GPUs - arXiv, fecha de acceso: mayo 11, 2026, <https://arxiv.org/html/2502.18403v1>
15. Optimizing WRF physics for multi-decadal simulation of near-surface climate over arid Xinjiang, China - EGU sphere, fecha de acceso: mayo 11, 2026,

- <https://egusphere.copernicus.org/preprints/2026/egusphere-2025-6248/>
16. The kinetic preprocessor KPP*/a software environment for solving chemical kinetics, fecha de acceso: mayo 11, 2026,
<https://www2.mmm.ucar.edu/wrf/WG2/GPU/KPP-sdarticle.pdf>
 17. GPU Acceleration of a Chemistry Kinetics Solver, fecha de acceso: mayo 11, 2026,
<https://www2.mmm.ucar.edu/wrf/WG2/GPU/Chem.htm>
 18. GPU accelerated atmospheric chemical kinetics in the ECHAM/MESSy (EMAC) Earth system model (version 2.52) - GMD, fecha de acceso: mayo 11, 2026,
<https://gmd.copernicus.org/preprints/gmd-2017-63/gmd-2017-63.pdf>
 19. UC Riverside - eScholarship.org, fecha de acceso: mayo 11, 2026,
<https://escholarship.org/content/qt8s28g07q/qt8s28g07q.pdf>
 20. Automatic Code Rewriting for Performance Portability by Alister Johnson A dissertation accepted and approved in partial fulfillment - Computer Science - University of Oregon, fecha de acceso: mayo 11, 2026,
<https://www.cs.uoregon.edu/Reports/PHD-202403-Johnson.pdf>
 21. Parallel Computing: Architectures, Algorithms and Applications - JuSER, fecha de acceso: mayo 11, 2026,
<https://juser.fz-juelich.de/record/60553/files/NIC225296.pdf>
 22. (PDF) Acceleration of the Parameterization of Unified Microphysics Across Scales (PUMAS) on the Graphics Processing Unit (GPU) With Directive-Based Methods - ResearchGate, fecha de acceso: mayo 11, 2026,
https://www.researchgate.net/publication/370932928_Acceleration_of_the_Parameterization_of_Unified_Microphysics_Across_Scales_PUMAS_on_the_Graphics_Processing_Unit_GPU_With_Directive-Based_Methods
 23. US Department of Energy, Office of Science, High-Performance Computing Facility 2023 Operational Assessment Oak Ridge Leadership, fecha de acceso: mayo 11, 2026,
<https://www.olcf.ornl.gov/wp-content/uploads/OLCF-2023-OAR-Report-FINAL.pdf>
 24. E3SM C++ Atmosphere Model Our journey to Exascale - DOE Office of Science, fecha de acceso: mayo 11, 2026,
https://science.osti.gov/-/media/ascr/ascac/pdf/meetings/202309/Bertagna_ASCA_C_092823_EAMxx.pdf
 25. Exascale and artificial intelligence: a great marriage - Multicore World, fecha de acceso: mayo 11, 2026,
https://multicore.world/wp-content/uploads/2024/02/kothe-multicore-world_final_2.12.24.pdf
 26. E3SM-MMF Experiences with AMD Hardware - Oak Ridge Leadership Computing Facility, fecha de acceso: mayo 11, 2026,
<https://www.olcf.ornl.gov/wp-content/uploads/2021/01/2021UM-Day-3-Norman-E3SM-MMF-Experiences-with-AMD-Hardware.pdf>
 27. Pace v0.2: a Python-based performance-portable atmospheric model, fecha de acceso: mayo 11, 2026,
<https://www.research-collection.ethz.ch/server/api/core/bitstreams/b74fe24c-5c11-4511-8f86-cb51ba38c716/content>

28. Pace v0.1: A Python-based Performance-Portable Implementation of the FV3 Dynamical Core - ResearchGate, fecha de acceso: mayo 11, 2026, https://www.researchgate.net/publication/363931636_Pace_v01_A_Python-based_Performance-Portable_Implementation_of_the_FV3_Dynamical_Core
29. Fortran code on GitHub - Fortran - VASPKit论坛 - 手机版 - Powered by Discuz!, fecha de acceso: mayo 11, 2026, <http://bbs.vaspkit.cn/thread-71-1-1.html>
30. Area Exam: General-Purpose Performance Portable Programming Models for Productive Exascale Computing - Computer Science - University of Oregon, fecha de acceso: mayo 11, 2026, <https://www.cs.uoregon.edu/Reports/AREA-202006-Johnson.pdf>
31. Automatic Port to OpenACC/OpenMP for Physical Parameterization in Climate and Weather Code Using the CLAW Compiler | Supercomputing Frontiers and Innovations, fecha de acceso: mayo 11, 2026, <https://superfri.org/index.php/superfri/article/view/285>
32. Porting Applications with CUDA Fortran and OpenACC, fecha de acceso: mayo 11, 2026, <https://www.olcf.ornl.gov/wp-content/uploads/2012/05/Porting-Applications-with-CUDA-Fortran-and-OpenACC.pdf>
33. Fortran Programmers : How do you want to offload to GPU accelerators in the next 5 years?, fecha de acceso: mayo 11, 2026, <https://fortran-lang.discourse.group/t/fortran-programmers-how-do-you-want-to-offload-to-gpu-accelerators-in-the-next-5-years/247>
34. AI-Enhanced Decision Support for Photovoltaic Installations: from Rooftop Data Processing to Techno-Economic Prioritization, fecha de acceso: mayo 11, 2026, https://thesis.unipd.it/retrieve/de9e6272-be36-4a79-82d4-3545186c9e80/Zanon_Tommaso.pdf
35. Correctness and Verification - ASAP, fecha de acceso: mayo 11, 2026, <https://asap.ucar.edu/projects/verify/>
36. Nine time steps: ultra-fast statistical consistency testing of the Community Earth System Model (pyCECT v3.0) - GMD, fecha de acceso: mayo 11, 2026, <https://gmd.copernicus.org/articles/11/697/2018/>
37. A new ensemble-based consistency test for the Community Earth System Model (pyCECT v1.0) | Semantic Scholar, fecha de acceso: mayo 11, 2026, <https://pdfs.semanticscholar.org/c965/43226fd1ce45dc18ac85d49776e0b0d88abf.pdf>
38. The ensemble consistency test: from CESM to MPAS and beyond - GMD - Copernicus.org, fecha de acceso: mayo 11, 2026, <https://gmd.copernicus.org/articles/18/2349/2025/>
39. The ensemble consistency test: From CESM to MPAS and beyond | Mesoscale & Microscale Meteorology, fecha de acceso: mayo 11, 2026, <https://www.mmm.ucar.edu/opensky-publications/articles:43521/the-ensemble-consistency-test-from-cesm-to-mpas-and-beyond>
40. The ICON-A model for direct QBO simulations on GPUs (version icon-cscs:baf28a514), fecha de acceso: mayo 11, 2026, <https://gmd.copernicus.org/articles/15/6985/2022/>

41. GitHub - Beliaevsky/Fortran-Tools: Links to Fortran compilers, preprocessors, formatters, static analyzers, transpilers, IDEs, build systems, etc., fecha de acceso: mayo 11, 2026, <https://github.com/Beliaevsky/Fortran-Tools>
42. Model Releases - cosmo-model.org, fecha de acceso: mayo 11, 2026, <https://www.cosmo-model.org/content/model/cosmo/development/releases.htm>
43. The ICON-A model for direct QBO simulations on GPUs (version icon-cscs:baf28a514), fecha de acceso: mayo 11, 2026, https://www.researchgate.net/publication/363612422_The_ICON-A_model_for_direct_QBO_simulations_on_GPUs_version_icon-cscsbaf28a514
44. Toward Trustworthy Short-Range Forecasts with AFNO: From Skill Metrics to Conservation Checks - ResearchGate, fecha de acceso: mayo 11, 2026, https://www.researchgate.net/publication/400002509_Toward_Trustworthy_Short-Range_Forecasts_with_AFNO_From_Skill_Metrics_to_Conservation_Checks
45. Advanced Long-term Earth System Forecasting - arXiv, fecha de acceso: mayo 11, 2026, <https://arxiv.org/html/2505.19432v2>
46. METplus | dtcenter.org, fecha de acceso: mayo 11, 2026, <https://dtcenter.org/community-code/metplus>
47. Arxiv今日论文| 2026-02-12 - 闲记算法, fecha de acceso: mayo 11, 2026, http://lonepatient.top/2026/02/12/arxiv_papers_2026-02-12
48. (PDF) Km-scale climate simulations over Madeira and Canary Islands under present and future conditions: a model intercomparison study - ResearchGate, fecha de acceso: mayo 11, 2026, https://www.researchgate.net/publication/388073231_Km-scale_climate_simulations_over_Madeira_and_Canary_Islands_under_present_and_future_conditions_a_model_intercomparison_study
49. NVIDIA Earth-2 Powers Regional AI Weather Forecasting in the United Arab Emirates, fecha de acceso: mayo 11, 2026, <https://developer.nvidia.com/blog/nvidia-earth-2-powers-regional-ai-weather-forecasting-in-the-united-arab-emirates/>